INDEX

NAME: Jeevan Prabu STD.: IT SEC.: 'B' ROLL NO.: (24[illegible]15)

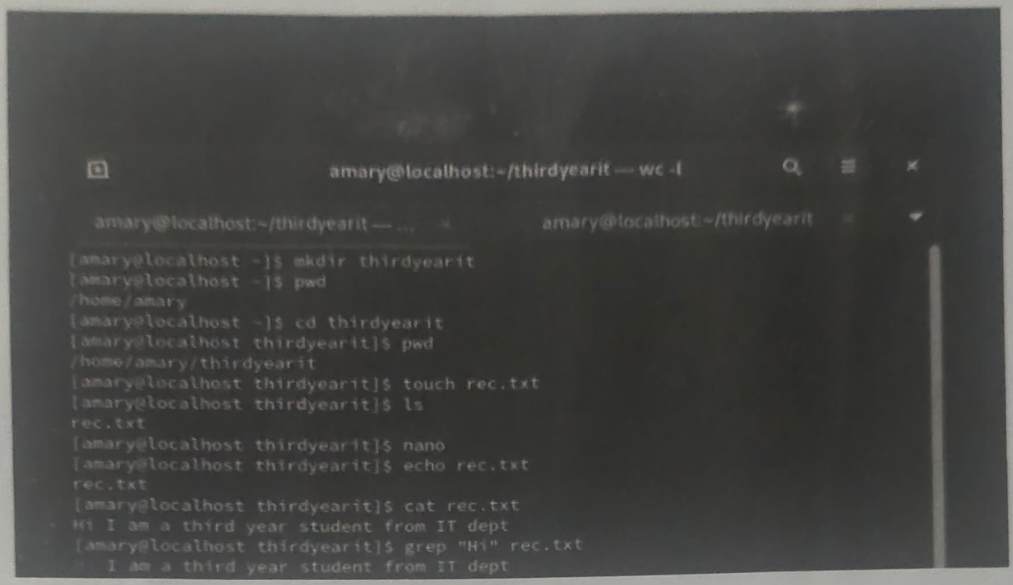
S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
1.	15/7/2026	Basic networking commands in linux and windows operating System		
2.	16/7/2026	Study of networking Cables		
3.	18/7/2026	Ping Command		
4.	22/7/2026	Packet tracer Tool		
5.	30/7/2026	Echo using python		
6.	30/7/2026	Byte Stuffing using python.		
7.	19/8/2026	Implementations of Subnetting in Cisco packet tracer simulator		
8.	9/9/26	Client-Server Communication using python socket programming		
9.	16/9/26	distance vector Routing		
10.	23/9/26	deploying a web application on AWS server		
11.	20/8/26	Experiment on packet Captures tool		

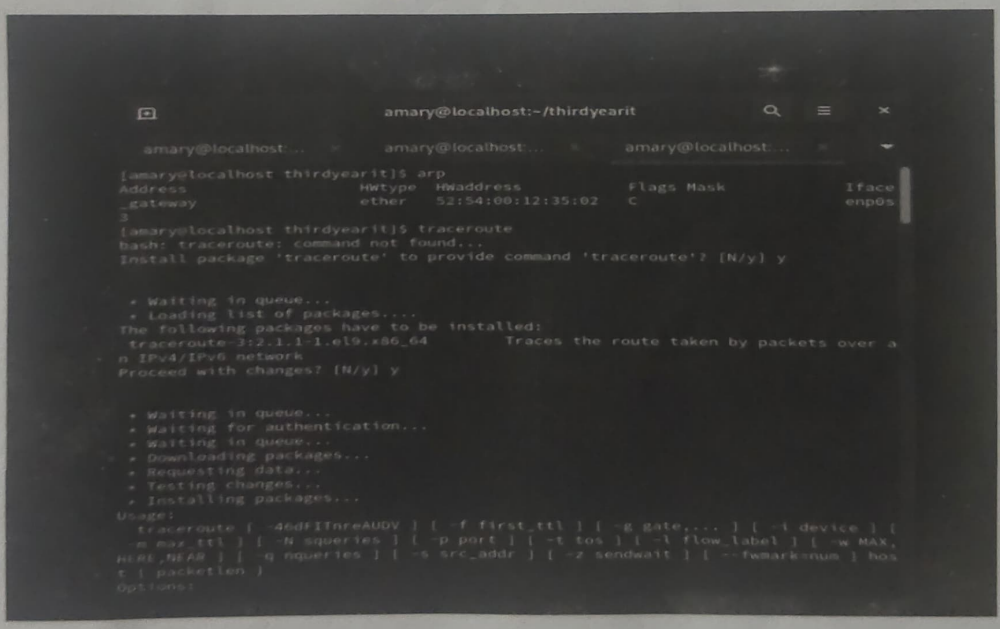
Ex.no:1

Date: 15/7/2026

BASIC NETWORKING COMMANDS IN LINUX AND WINDOWS OPERATING SYSTEM

Aim:
To basic networking commands in linux and windows operating system





Result:

Thus the basic networking commands in Linux and windows operating system has been completed

EXP No:2 Learning and Assignment of IP
DATE: 16/7/26 Address Manually to Compulsory.

AIM:
To study the different types of network Cables

a) Understanding different types of network
1) unsheild twisted Pair (UTP) Cable)
2) Shielded twisted Pair (UTP) Cable
3) Coaxial Cable
4) Fibre optic Cable

Cable Type	Category	Maximum Data transmission	Advantage / Disadvantage	Application
UTP	Category 3 Category 5 Category 5e	10bps upto 100 Mbps 1 Gbps	Advantage → Cheaper → Easy to install Disadvantage → more prone to EMI & noise	10 Base Techanet Fast Ethernet
STP	Category 6a	10 Gbps	* Shielded * Faste than UTP	Gigabit Ethernet
Coaxial Cable	RG-6 RG-59 RG-11	10-100 Mbps	* High band width * Versatile Disadvantage * Limited distance	Speed of Signal is 500m Television network

Result:

Thus, the different types of network Cables are studied and verified.

Ex.no:3 DEVELOP A CUSTOMIZED PING COMMAND

Date: 18/7/2026 TO TEST THE SERVER CONNECTIVITY

AIM: To develop a customized ping command to test the server connectivity

Command Prompt

```
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Tcs>ping google.com

Pinging google.com [142.250.205.110] with 32 bytes of data:
Reply from 142.250.205.110: bytes=32 time=9ms TTL=120
Reply from 142.250.205.110: bytes=32 time=6ms TTL=120
Reply from 142.250.205.110: bytes=32 time=17ms TTL=120
Reply from 142.250.205.110: bytes=32 time=15ms TTL=120

Ping statistics for 142.250.205.110:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 6ms, Maximum = 17ms, Average = 11ms

C:\Users\Tcs>nslookup google.com
Server:  UnKnown
Address:  172.16.8.1

Non-authoritative answer:
Name:    google.com
Addresses:  2404:6800:4007:807::200e
          142.251.221.174


C:\Users\Tcs>ping 172.16.8.1

Pinging 172.16.8.1 with 32 bytes of data:
Reply from 172.16.8.1: bytes=32 time<1ms TTL=64
Reply from 172.16.8.1: bytes=32 time<1ms TTL=64
Reply from 172.16.8.1: bytes=32 time<1ms TTL=64
Reply from 172.16.8.1: bytes=32 time<1ms TTL=64

Ping statistics for 172.16.8.1:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 0ms, Average = 0ms

C:\Users\Tcs>_
```

RESULT:

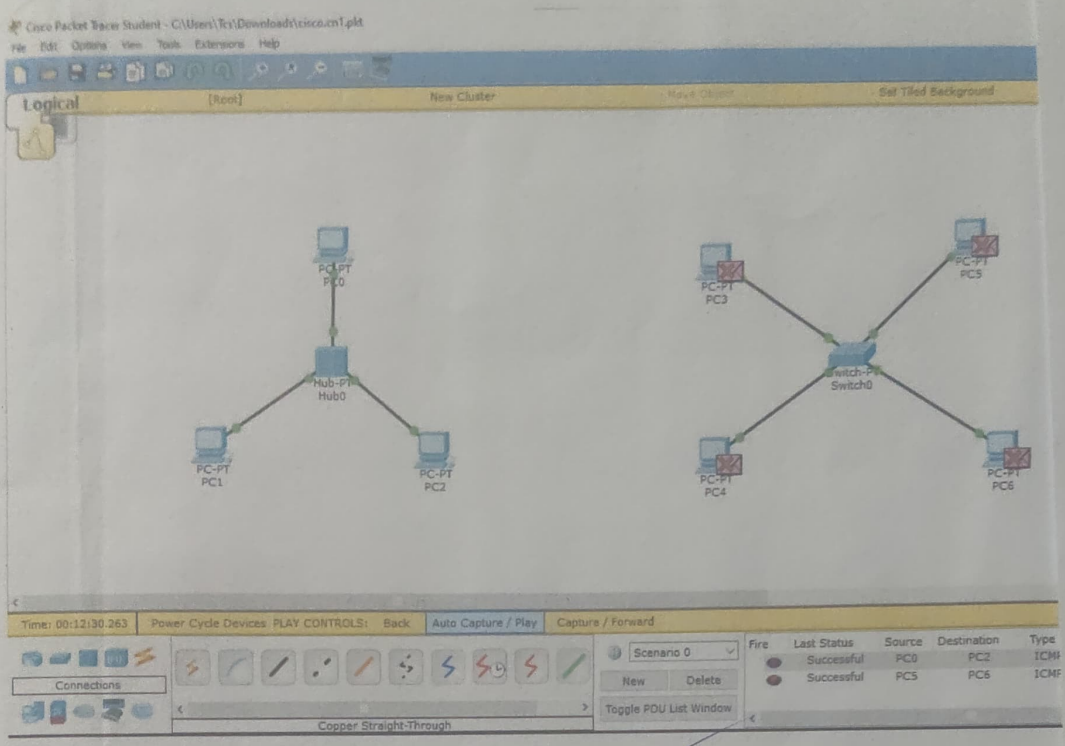
Thus the ping command to test the server connectivity is developed successfully.

Exp no: 4 PACKET TRACER TOOL INSTALLATION

Date: 22.7.26 AND USER INTERFACE OVERVIEW

Aim:

To instance cisco packet tracer and study its user interface and basic features



RESULT:

Thus, the packet tracer tool installation and user interface overview is executed successfully.

Ex.no: 5

30.7.26

IMPLEMENT ECHO USING PYTHON

Aim:

To Implement Echo using python programming.

```
import socket

HOST = '127.0.0.1'   # Localhost
PORT = 12345

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind((HOST, PORT))
server.listen(1)

print("Server is waiting for connection...")

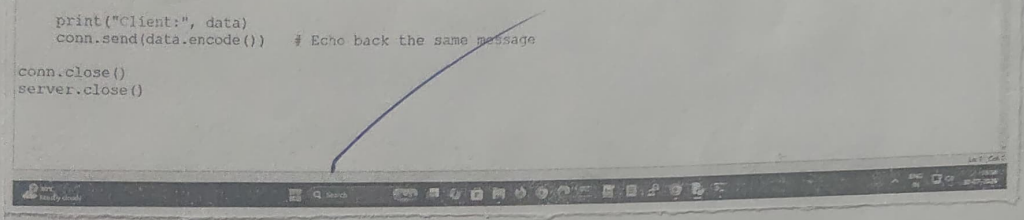
conn, addr = server.accept()
print("Connected by:", addr)

while True:
    data = conn.recv(1024).decode()

    if not data or data.lower() == "exit":
        print("Connection closed.")
        break

    print("Client:", data)
    conn.send(data.encode())   # Echo back the same message

conn.close()
server.close()
```



```
import socket

HOST = '127.0.0.1'
PORT = 12345

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect((HOST, PORT))

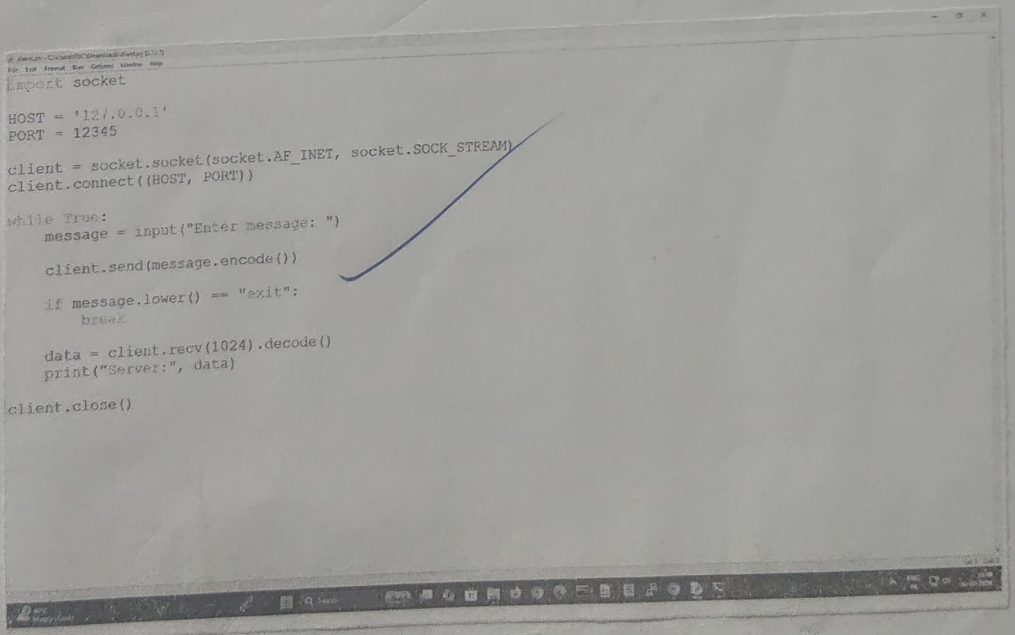
while True:
    message = input("Enter message: ")

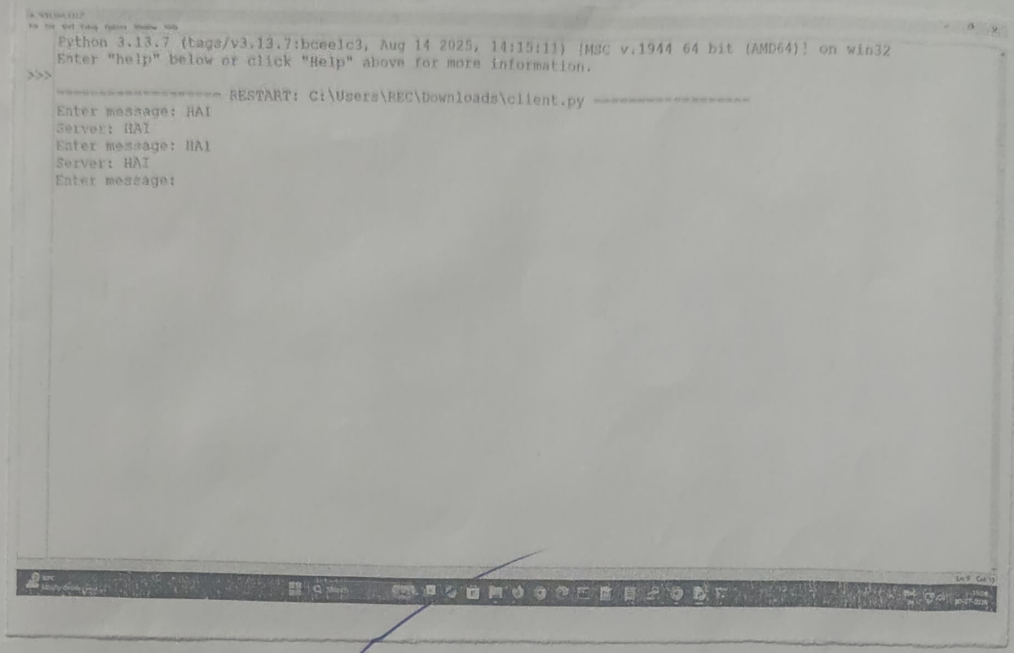
    client.send(message.encode())

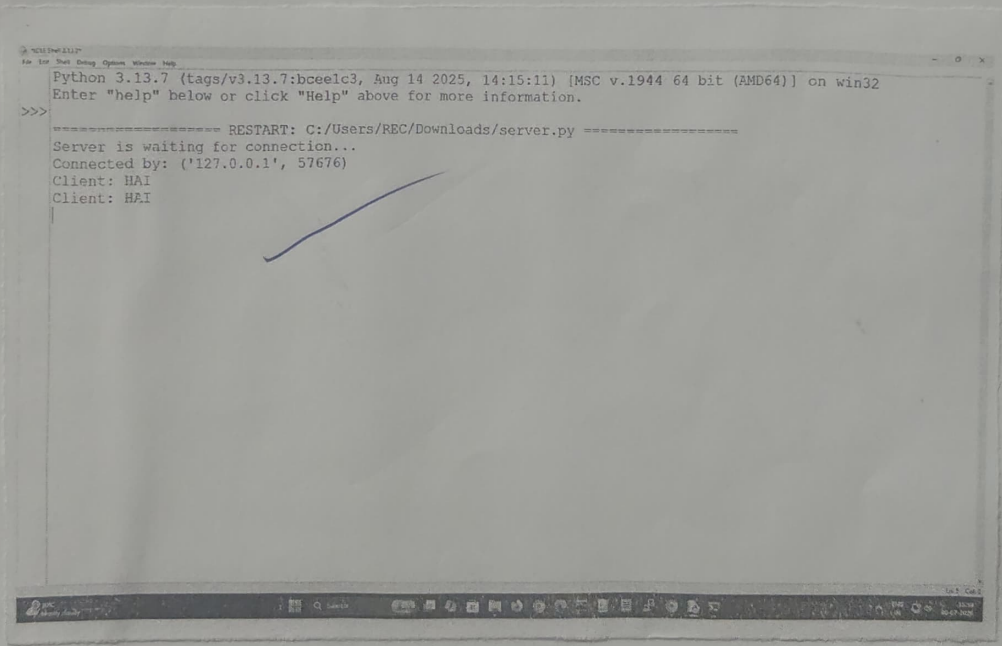
    if message.lower() == "exit":
        break

    data = client.recv(1024).decode()
    print("Server:", data)

client.close()
```







RESULT:

Thus the implementation of echo using server and client is completed.

Date : 30.7.26

Exp. No: 6

BYTE STUFFING USING PYTHON

Aim:

To implement byte stuffing using Python programming.

byte_stuffing.py - C:/Users/REC/AppData/Local/Programs/Python/Python310/byte_stuffing.p...

File Edit Format Run Options Window Help

```
# Byte Stuffing with Binary Representation

flag = input("Enter Flag Character: ")
escape = input("Enter Escape Character: ")
data = input("Enter Data: ")

stuffed = ""

for ch in data:
    if ch == flag or ch == escape:
        stuffed += escape
    stuffed += ch

frame = flag + stuffed + flag

print("\nOriginal Data      :", data)
print("Stuffed Data       :", stuffed)
print("Transmitted Frame  :", frame)

# Function to print binary values
def show_binary(text):
    for ch in text:
        print(ch, "->", format(ord(ch), '08b'))

print("\nBinary of Flag:")
show_binary(flag)

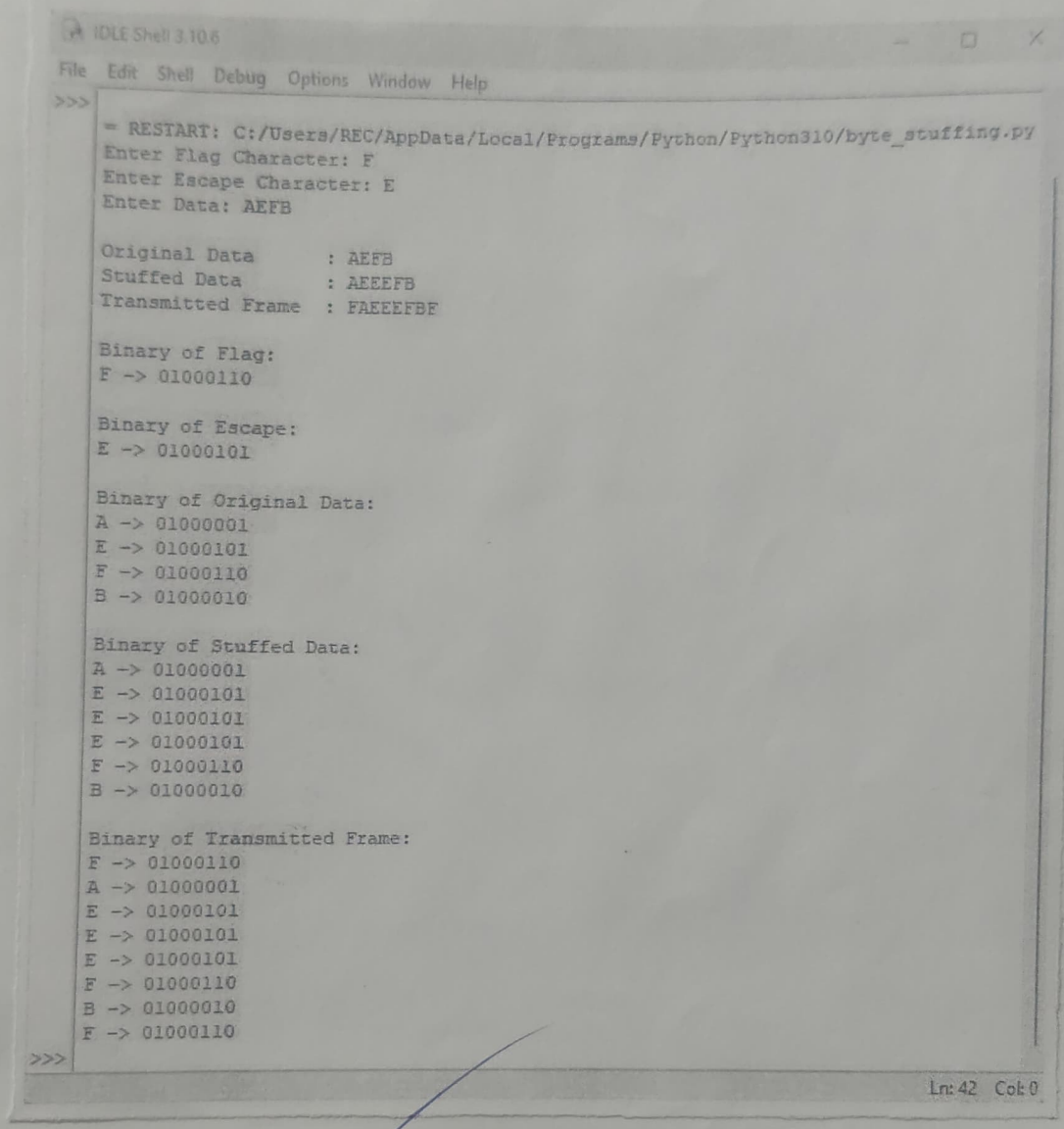
print("\nBinary of Escape:")
show_binary(escape)

print("\nBinary of Original Data:")
show_binary(data)

print("\nBinary of Stuffed Data:")
show_binary(stuffed)

print("\nBinary of Transmitted Frame:")
show_binary(frame)
```

Ln: 39 Col: 0



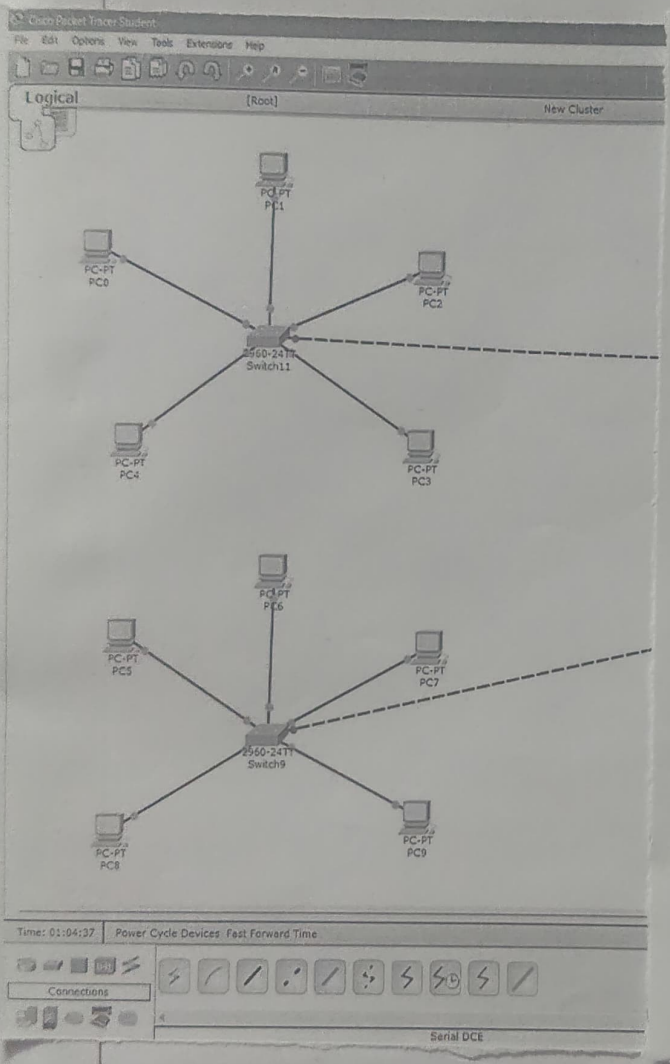
RESULT:

Thus the implement byte stuffing using the Python programming is completed.

Ex.no:7 Implementation of Subnetting in
19/8/26 CISCO PACKET TRACER Simulator

AIM:

To implement the Subnetting in the CISCO PACKET TRACER Simulator



Result:

Thus, Cisco packet Tracer Simulator is implement Successfully.

Ex. No 8 Client Server Communication using Python Socket Programming

9/9/2026

Aim:

The client Server Communication using Python Socket programming

client.py - C:/Users/REC/client.py (3.12.10)

File Edit Format Run Options Window Help

```
import socket
import threading

SERVER_IP = "127.0.0.1"
PORT = 3001


def receive_messages(sock):
    while True:
        try:
            message = sock.recv(1024).decode()

            if not message:
                break

            print(f"\nServer: {message}")
            print("You: ", end="", flush=True)

        except ConnectionResetError:
            break


client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect((SERVER_IP, PORT))

print("Connected to server.")

threading.Thread(
    target=receive_messages,
    args=(client,),
    daemon=True
).start()

while True:
    message = input("You: ")

    if message.lower() == "exit":
        break

    client.sendall(message.encode())

client.close()
```

server.py - C:/Users/REC/server.py (3.12.10)

File Edit Format Run Options Window Help

```
import socket
import threading

HOST = "0.0.0.0"
PORT = 3001


def receive_messages(conn):
    while True:
        try:
            message = conn.recv(1024).decode()

            if not message:
                break

            print(f"\nClient: {message}")
            print("You: ", end="", flush=True)

        except ConnectionResetError:
            break


server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind((HOST, PORT))
server.listen(1)

print(f"Server listening on port {PORT}...")

conn, addr = server.accept()
print(f"Client connected: {addr}")

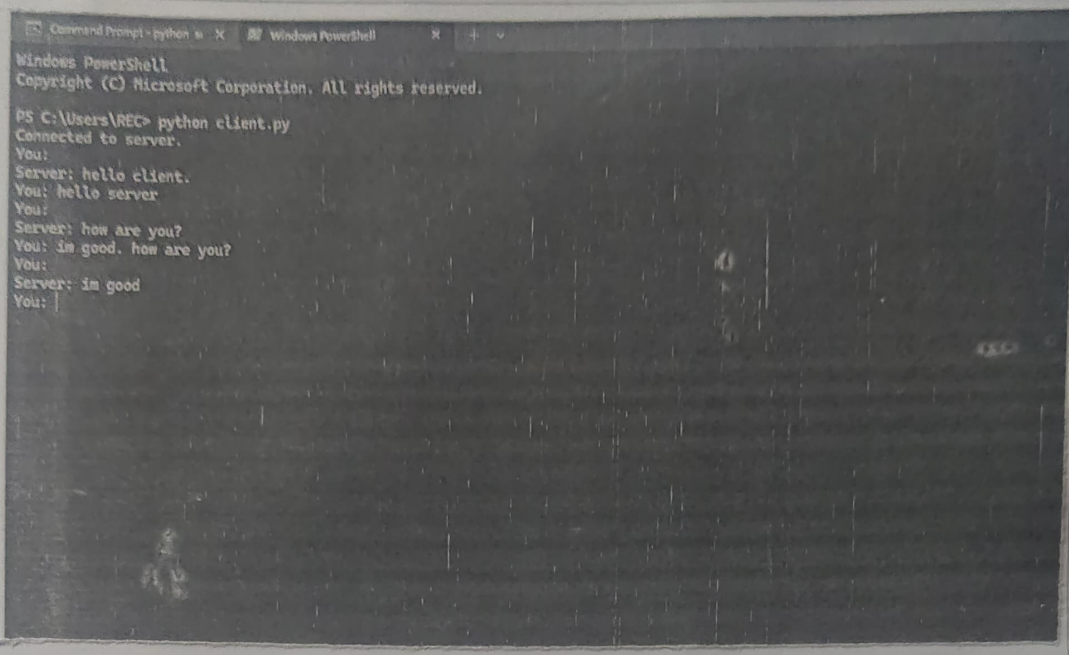
threading.Thread(
    target=receive_messages,
    args=(conn,),
    daemon=True
).start()

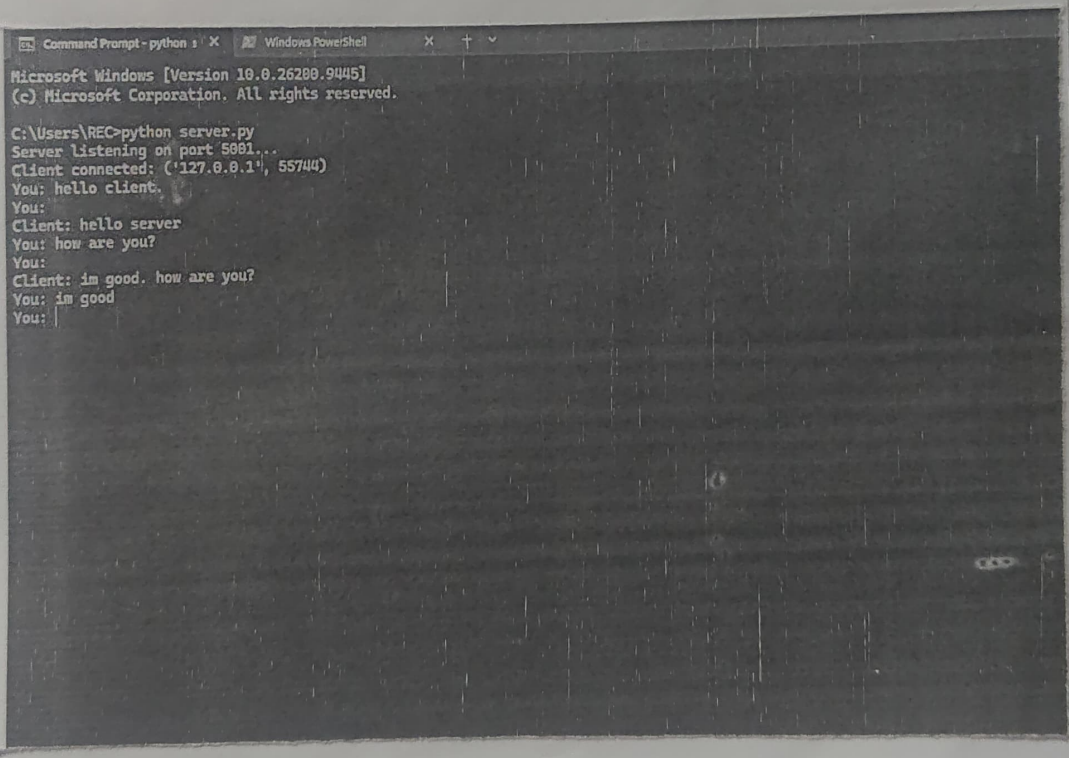
while True:
    message = input("You: ")

    if message.lower() == "exit":
        break

    conn.sendall(message.encode())

conn.close()
server.close()
```





Result:

Thus the client ~~Circuit~~ Server Communication was successfully established and message was exchanged between client and Server.

Command Prompt - python

```
...     net.display_tables()

Simulating Distance Vector Routing Protocol...
Network converged in 3 iterations.

--- Routing Table for Router A ---
Destination    Cost      Next Hop
A              0         A
B              2         B
C              3         B
D              5         B

--- Routing Table for Router B ---
Destination    Cost      Next Hop
A              2         A
B              0         B
C              1         C
D              3         C

--- Routing Table for Router C ---
Destination    Cost      Next Hop
A              7         A
B              1         B
C              0         C
D              2         D

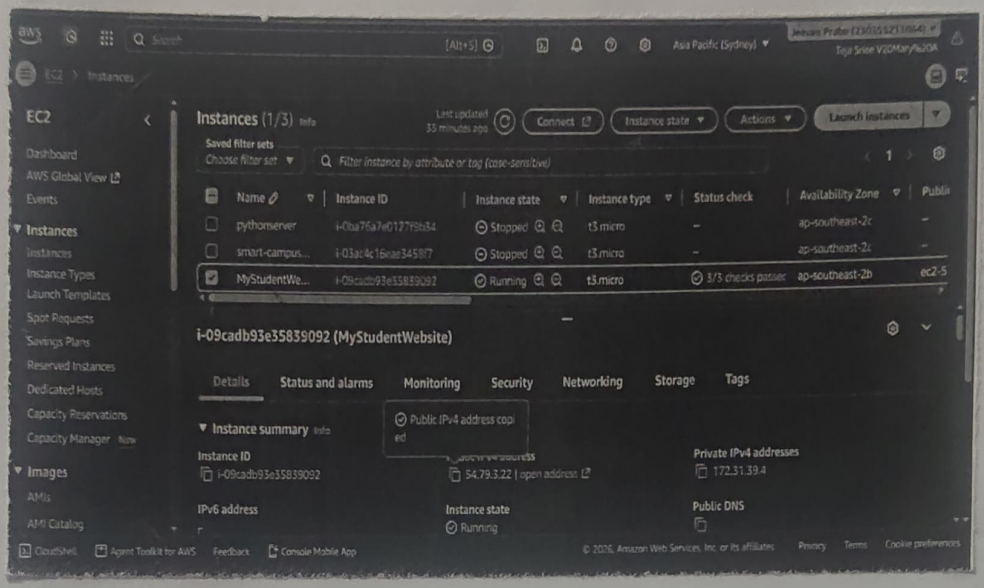
--- Routing Table for Router D ---
Destination    Cost      Next Hop
A              inf       -
B              4         B
```

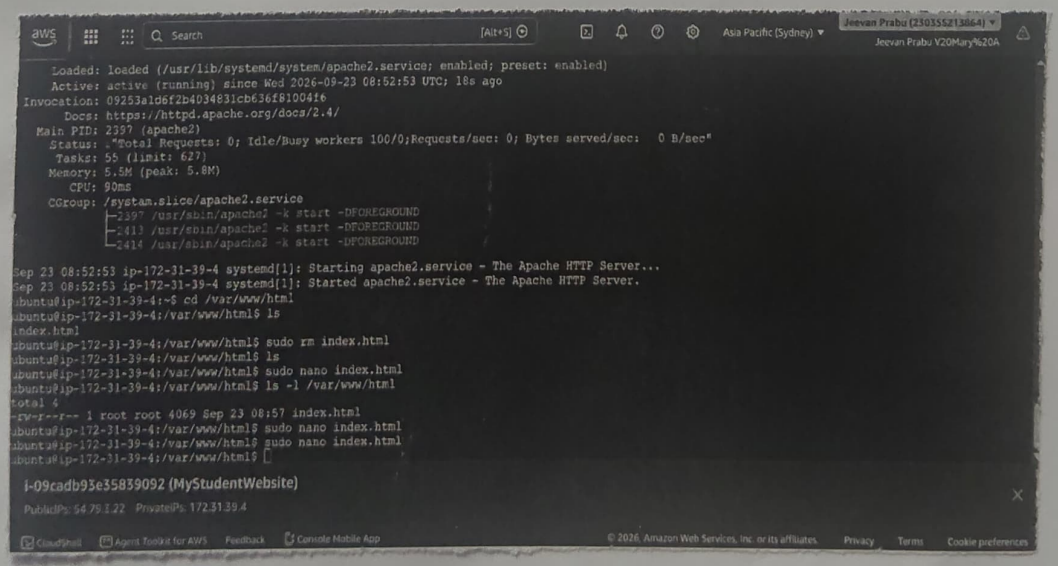
Result:

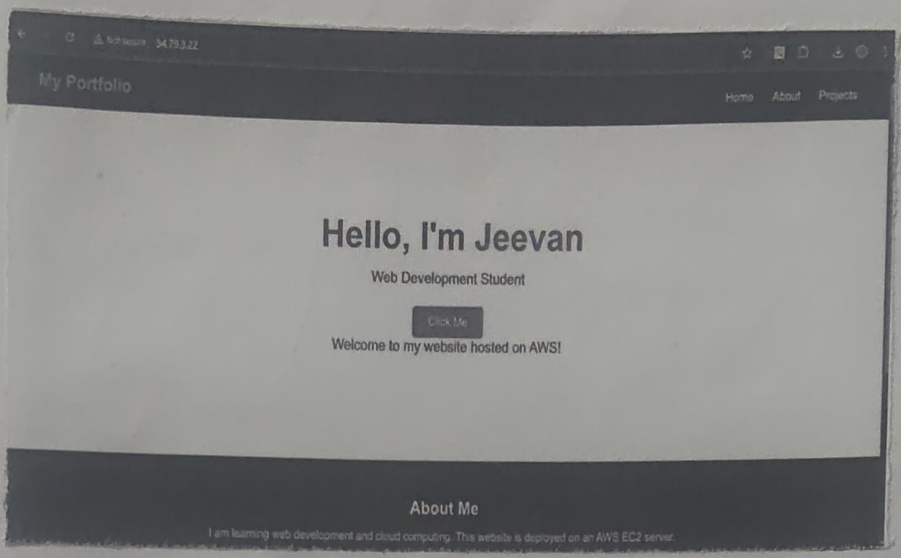
Thus, the implementation of Distance vector routing protocol of networks converged in 3 iterations has been successfully compiled.

Aim:

To Deploying a web application on AWS Server







Result:

Thus, the Deployment of web application on AWS server has been Deployed successfully.

Ex.no : 11 EXPERIMENT ON PACKET CAPTURE

20/8/26 TOOL - WIRESHARK

AIM:

Capturing an analysis packets using wire-shark tool.

1) AIM:

Create a filter to Display only TCP/UDP packets, inpects the packets and provide the flow graph.

PROCEDURE:

* Select LAN Connection in wireshark
* Go to Capture - option.
* Select stop Capture automating after 100 packets
* Save the packets.

2) AIM:

Create a filter to Display only ARP packets and inspect the packets.

PROCEDURE:

* Go to Capture
* Set stop Capture automatically after 100 packets
* Then click start capture
* Save the packets

3) AIM:

Create a filter to display only DNS packets and provide flow graph.

PROCEDURE:

* Go to Capture -> option
* Selet stop Capture automatingically after 100 packets.

* Then click start Capture
* Save the packets.

4) AIM:
Create a filter to display only HTTP packet & repeat the packets.

PROCEDURE:
* Select LAN Connections in wireshark.
* Go to ~~start~~ Capture → option
* click start Capture.
* Search for ICMP/IP packets.
* Save the packets.

5) AIM:
To create a filter to display only SMCP packets and inspect the packets.

PROCEDURE:
* Select LAN Connections in wireshark.
* Go to Capture → option
* Select the stop Capture automatically.
* Save the packets.

Result:
Thus, network packets were successfully captured and analysed using wireshark with different protocal filters.

Ex.no: 12 Discovery of live host using ARP, ICMP, TCP, UDP.

20/9/26

AIM:

To discover live host in a network using Nmap scans such as ARP, ICMP, TCP & UDP on the TryHackme platform.

OBJECTIVE:

To understand how nmap identifies active systems before performing further network scanning.

SOFTWARE:

Nmap
Tryhack Nmap learn
Computer with network success

THEORY:

Nmap is a free and open-source network scanning tool used to discover live hosts and identify network services. It was created.

Nmap uses different live hosts using different protocols.

1) ARP Scan → Uses ARP scans request to find hosts on same subnet.

2) ICMP scan ⇒ USE Icmp request and replies to identify active host

3) TCP Scan - Use TCP requests and packets to specified ports & check the response.

OBSERVATION:

* ARP request can discover hosts within the same subnet
* ARP cannot cross routers because it operates at the link layer.
* AN ARP ping generally requires a privileged account.
* UDP ping can identify an active host when an ICMP port unreachable is received.

RESULT:

Thus, the live host discovery using Nmap was successfully studied and performed. The experiment helped in understanding.

Ex. No: 13
27/8/26

SLIDING WINDOW PROTOCOL

AIM:

To implement the flow control at the data link layer using the sliding window protocol and simulate the flow of frames from one node to another.

SENDER CODE:

```
message = input ("Enter the message:")
window_size = int (input ("Enter window size:"))
frames = []
for i, ch in enumerate (message):
    frames.append ((i+1, ch))
print (f"Frame {no}: {data}")
with open ("Sender_Buffer.txt", "w") as file:
    for no, data in frames:
        file.write (f"{no} {data} \n")
print ("In current window:")
for i in range (base, min (base + window
                    - size, len (frames))):
    print (f"[{frames[i][0]}: {frame[i][1]}
                    ]", end = " ")

try:
    with open ("Receiver_Buffer.txt", "r") as
                    file:
        response = file.readline().strip()
```

```
except file Not found Error:
    response = " "
if response.starts with ("ACK"):
    ack = int(response.split()[1])
    print(f"Ack {ack} received")
    base = ack
    open("Receiver_Buffer.txt", "w").
                close()
elif response.starts with ("NACK"):
    nack = int(response.split()[1])
    print(f"Retransmitting frame {nack}
    open("Receiver_Buffer.txt", "w") close()
else:
    print("\n Frames sent")
    input("Run receiver program & press
            ENTER to CONTINUE")
    print("ALL frames transmitted")
```

RECEIVER CODE:

```
While True:
    try:
    with open("Sender_Buffer.txt", "r") as
                file:
        frames = file.readlines()
except file Not Found Error:
    print("Sender_Buffer.txt not found")
    break
```

INPUT:

Enter the message : HELLO

Enter Window Size : 3

OUTPUT:

Frames Created

Frame 1 : H

Frame 2 : E

Frame 3 : L

Frame 4 : L

Frame 5 : O

Frames stored in Sender_Buffer.txt

Current window:

[1:H] [2:E] [3:L]

Frames Sent

Run receiver program and press Enter to Continue

RECEIVER OUTPUT:

Frames received from Sender:

Frame 1 : H

Frame 1 received Correctly.

ACK 1 sent

RESULT:

The Sliding window protocol was successfully implemented.

Exp No: 14
8/10/26

IMPLEMENTING PACKET SNIFFING USING RAW SOCKETS

AIM:

To implement packet sniffing using RAW SOCKET in Python.

ALGORITHM:

- Create a raw socket
- Bind the socket to the network interface
- Receive packet continously.
- Display the source & destination IP add
- Continue until the user stop the program.

INPUT:

Network packets received through the interface

CODE:

```
import socket
S= socket.socket (socket.AF_INET, socket.
                  SOCK_RAW.SOCKET.IPPROTO_IP)
S.bind (("0.0.0.0", 0))
S.setsockopt (socket.IPPROTO_IP, socket.
              IP_HDR INCL 1)
print ("PACKET Sniffer Started...")
while True:
packet, addr = s.recv from (65565)
    ip_header =packet [0:20]
    iph = struct.unpacket ("!BBHHHBBH4S
                           4s", ipheader).
```

```
src_ip = socket.inet_ntoa(iph[8])
dest_ip = socket.inet_ntoa(iph[9])
print("Source IP:", src_ip)
print("Destination IP:", dest_ip)
print("Packet Length:", len(packet))
```

OUTPUT:

```
Packet Sniffer started....
Source IP: 192.168.1.10
Destination IP: 192.168.1.1
Packet length : 79.
```

RESULT:

Thus a raw Socket program was Successfully implemented.

Exno: 15 ANALYSE VARIOUS TYPES OF SERVER
8/10/26 LOGS USING WEBALIZER TOOL

AIM:
To analyze various types of server logs using webalizer tool.

ALGORITHM:
1. Install and open webalizer
2. Select the web server log file
3. Configure webalizer to read the log file
4. Run web to process the logs
5. Open the generated HTML report
6. Analyse HTTP status, visitor, request pages.

INPUT:
The web server log file such as access log

OUTPUT:
webalizer generated report
* Number of visitor
* Number of request
* Most visited pages
* Data transferred
* HTTP status
* Browser & operating system.

RESULT:
The web logs were successfully verified.